

PRACTICAL NO.: 4

Experiment Title: Use Function Types, Scope, and Closures; Apply Try-Catch; Build a Palindrome Checker.

Software/ Tools Required: VS Code (text editor), HTML, CSS, JavaScript.

Theory

JavaScript is a powerful scripting language used to create interactive and dynamic web applications. It works with HTML and CSS to provide functionality such as user interaction, input validation, event handling, and dynamic content updates. JavaScript allows developers to write reusable and efficient code using functions, variable scope, closures, and exception handling.

Functions are one of the fundamental building blocks of JavaScript. Different types of functions such as function declarations, function expressions, arrow functions, and anonymous functions are used to perform specific tasks. Functions make programs modular, reusable, and easier to maintain.

Another important concept is **scope**, which determines the visibility and accessibility of variables within different parts of a program. JavaScript provides three main types of scope:

- **Global Scope:** Variables can be accessed from anywhere in the program.
- **Function Scope:** Variables declared inside a function are accessible only within that function.
- **Block Scope:** Variables declared using **let** and **const** are limited to the block in which they are declared.

JavaScript also supports **closures**, a feature where an inner function remembers and accesses variables from its outer function even after the outer function has completed execution. Closures are widely used for data privacy, maintaining state, callbacks, and creating reusable functions.

To improve program reliability, JavaScript provides **exception handling** using the **try-catch** statement. The **try** block contains code that may produce an error, while the **catch** block handles the error without terminating the program. This allows applications to display meaningful error messages and continue execution smoothly.

In this experiment, a modern web application named **Palindrome.Core** is developed using HTML, CSS, and JavaScript. The application accepts a word or number from the user and determines whether it is a palindrome. A palindrome is a sequence that reads the same forward and backward, such as **"nayan"**, or **"121"**. JavaScript functions are used to reverse the input and compare it with the original value. Input validation and **try-catch** exception handling ensure that invalid or unexpected inputs are handled gracefully. The

result is displayed dynamically on the webpage with a modern user interface, making the application interactive and user-friendly.

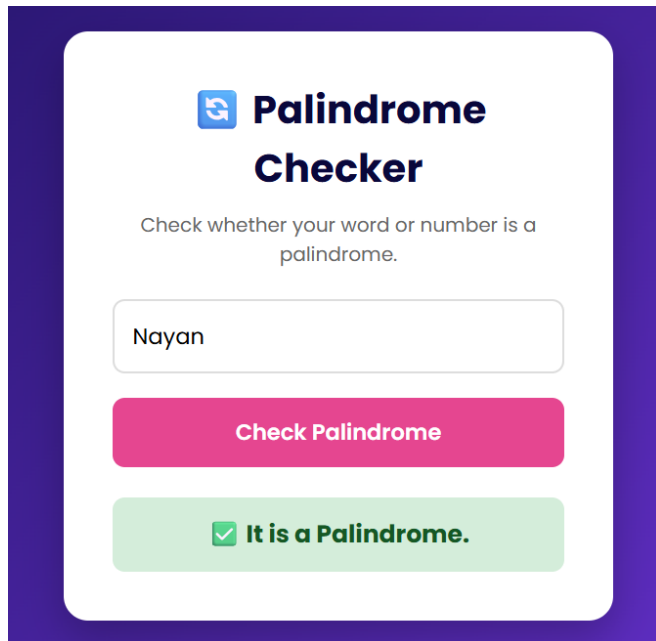
Output :

Case 1 (Palindrome)

Input: Nayan

Output →

VALID: "NAYAN" IS A PALINDROME



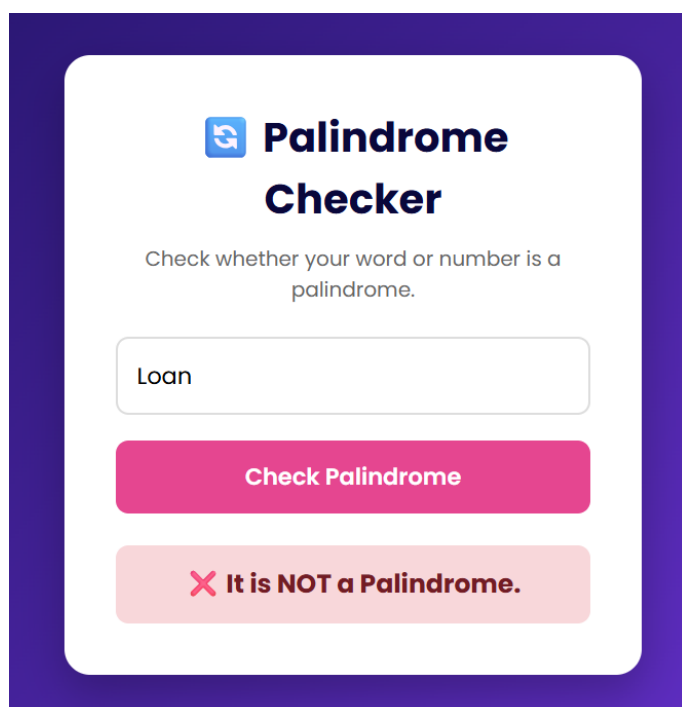
The screenshot shows a web application titled "Palindrome Checker" with a blue icon. Below the title is a subtitle: "Check whether your word or number is a palindrome." There is a text input field containing the word "Nayan". Below the input field is a pink button labeled "Check Palindrome". At the bottom, a green box displays the result: "✓ It is a Palindrome."

Case 2 (Not Palindrome)

Input: Loan

Output →

VALID: "LOAN" IS NOT A PALINDROME



The screenshot shows the same "Palindrome Checker" web application. The text input field now contains the word "Loan". The pink "Check Palindrome" button is still present. At the bottom, a pink box displays the result: "✗ It is NOT a Palindrome."

Result

The experiment was performed successfully. The Palindrome.Core application accepted user input, validated it correctly, and accurately determined whether the entered word or number was a palindrome. The program successfully displayed appropriate messages for both palindrome and non-palindrome inputs while handling unexpected errors using the **try-catch** mechanism.

Conclusion

The experiment was completed successfully. It demonstrated the implementation of JavaScript function types, variable scope, closures, and exception handling using **try-catch**. The Palindrome.Core application effectively validated user input, performed palindrome checking, and displayed results dynamically through an interactive web interface. This experiment enhanced the understanding of JavaScript programming concepts and modern client-side web application development.

Nayan Awalpane , Sem V-A, 24070521010